HARNESS LAB · MÓDULO 4 · AULA 4.3

Spec com *critério de aceite verificável*

"Faz o cadastro direito" não é spec. Spec é critério que dá sim ou não, escrito antes do código. O `/plan` pega a sua frase, questiona contra o código de hoje e devolve SPEC, plano e fases prontas para o loop.

`/plan` `specifier` `clarifier` `planner` `GEARS` `RIGID e FLEXIBLE` `AS IS e TO BE` `PHASES.md` `confere-phases.sh`

VOCÊ SAI COM

Os critérios da sua feature-alvo com ID e um teste cada, e o `/plan` e os três agentes da aula no `.claude/` do seu projeto, rodados nela: `.spec/features/<slug>/` com a SPEC de critérios binários, o PLAN e o PHASES que o loop do Módulo 5 vai executar.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

A aula promete um comando que gera spec, validações e fases para o loop do Módulo 5, e pede de volta o `FEATURE-ALVO.md` da 1.5, a base de tudo. "Faz o cadastro direito, valida o e-mail, mensagem amigável" não é especificação. Especificação é critério de aceite: **se o e-mail já existe com outra caixa, então o sistema deve recusar e não deve criar usuário.** Binário (sim ou não), com um estado final que se mede (`users` com uma linha e erro no campo e-mail), um jeito de provar e o que não pode mudar (o login segue funcionando).

FORMA	MOLDE	NO CADASTRO
Evento	QUANDO algo acontece, o sistema DEVE ...	QUANDO o e-mail é novo, DEVE criar o usuário
Estado	ENQUANTO num estado, o sistema DEVE ...	ENQUANTO o e-mail está em uso, DEVE recusar
Condição	SE ..., ENTÃO o sistema DEVE ...	SE recusar, ENTÃO DEVE mostrar a mensagem
Indesejado	SE ..., ENTÃO o sistema NÃO DEVE ...	SE o e-mail existe, ENTÃO NÃO DEVE criar usuário

Sem ambiguidade, sem gap

No legado, o buraco aparece depois. O agente que leu a documentação da 3.4 avisa: com a spec nova, vem este efeito colateral.

Um teste por critério

Dá para saber o que já foi feito e o que falta, voltar de onde parou e saber exatamente qual critério está falhando.

O */plan*: o orquestrador da chatice

Mesmo princípio do `/bc-harness:ai-context` da 3.4: um comando que orquestra subagentes. Entra o card do Jira ou do Trello, ou uma frase; quanto mais simples, mais perguntas. Chato de propósito, com o código de hoje como insumo.



Paradas: 1 critérios, slug e tier · 2 a SPEC · 3 as perguntas do clarifier · 4 as fases. Em cada uma, o /plan espera você.

O **especificador** ("senior Requirements Engineer") lê o código, confirma o que muda e escreve os requisitos nas formas da tabela (e na *Optional*, rara), com AS IS e TO BE. O **clarificador** cruza a spec com a regra de negócio e pega o bug que só aparece em produção. O **planejador** quebra em fases do tamanho do modelo: cada task com arquivos esperados (não na pedra), mudança, critério verificável e testes.

A demo, no blog da Beer and Code: `/plan Quero incluir um contador de visitas aos posts do blog`. Ele leu a arquitetura e as regras de domínio, perguntou onde exibir (só no admin) e se separa por idioma, e rascunhou os critérios: visita à página pública conta, crawler não, o mesmo visitante numa janela curta não conta de novo, e a contagem funciona com o post servido do cache. Vieram as ambiguidades (hash de IP, trusted proxies, janela de 24 horas, bots), conferidas pelo clarificador. O planner também pode perguntar, e no fim o `/plan` confere o formato dos arquivos em disco, como o `confere-phases.sh` faz fora da sessão. Saíram **10 tarefas em 5 fases**, com SPEC, PLAN, PHASES e `openapi.yaml`. No Módulo 5, cada task (janela de 200 mil tokens) ou cada fase (1 milhão) vira uma iteração do loop.

2 Indo *além da aula*

ALÉM DA AULA · O /PLAN DO PROJETO TROCA O /PLAN NATIVO

O Claude Code já tem um `/plan`, que entra no modo de planejamento. O seu `.claude/commands/plan.md` tem o mesmo nome e, num terminal local, **substitui o nativo**; o modo de planejamento segue no `Shift+Tab`. Só com o plugin BC Harness, o pipeline é `/bc-harness:plan` e o `/plan` puro continua o nativo. Commitou o `.claude/commands/plan.md`? O `/plan` muda para todo o time nesse repositório. Para não mexer no nativo, use só o `/bc-harness:plan`, sem a cópia.
Fonte: skills, "Resolve skills that share a name": "your skill replaces the built-in command".

ALÉM DA AULA · ANTES DE RODAR: AGENTS.MD E O FEATURE-ALVO.MD

Sem `AGENTS.md` nem `docs/agents/`, o `/plan` para e pergunta se segue sem arquitetura; se seguir, a SPEC sai sem as regras do seu projeto. Rode antes o `/bc-harness:ai-context` da 3.4. Com `/plan FEATURE-ALVO.md`, critério em lista é extraído; sem critério, ele rascunha de 3 a 7, marcados `(drafted)`. Confira se as "Cinco peneiras" não viraram critério. Até 3 critérios dá tier `light`: uma fase só. E peça a SPEC em português: na gravação, requisito saiu em inglês.
Fonte: `plan.md`, passos 1 e 4 e "Complexity tier"; SPEC real da gravação.

ALÉM DA AULA · O CLARIFICADOR RODA QUANDO VOCÊ PEDE

Ele só roda sozinho se a SPEC tem `[NEEDS CLARIFICATION]` ou se o tier é `complete`. Quer ser questionado? Diga na parada 2 que tem dúvida. A sugestão de cada pergunta é dele; a decisão é sua. Ele não questiona o FLEXIBLE (o como), só o RIGID (o que não se negocia). Fique por perto: com `askUserQuestionTimeout` ligado, pergunta sem resposta fecha sozinha e o Claude segue sem você. Um plano `standard` do Lucas levou cerca de 19 minutos.
Fonte: `plan.md`, passo 6; `clarifier.md`; tools-reference, "Question auto-continue timeout".

ALÉM DA AULA · O PHASES.MD É CONTRATO COM O LOOP

O loop do Módulo 5 corta o arquivo em `## Phase N: título` e roda cada fase numa sessão nova; outro `##` entre duas fases corta a de cima, calado. Toda task precisa de `Acceptance criteria:` e de `Testes:`. O `/plan` confere com `grep`; o `confere-phases.sh` confere fora da sessão e aponta a linha. Deu erro? Re-delegue ao planner: o PHASES é a vista 1:1 do PLAN. O `./ralph.sh` do resumo fica para o Módulo 5: sem `--engine claude` ele usa o Codex, e exige árvore limpa (passo 9).
Fonte: `planner.md`; `ralph.sh` do BC Harness 0.2.0. `confere-phases.sh` testado com seis defeitos.

ALÉM DA AULA · UM TESTE POR CRITÉRIO: ATIVO OU PENDENTE

O que já existe ganha teste **ativo**, verde. O que é novo, **pendente** com o corpo escrito: Pest `->todo()`; Jest e Vitest `test.skip` (o `test.todo` do Jest não aceita corpo); pytest `@pytest.mark.skip(reason=...)`. O loop roda a suíte inteira em toda fase: um ativo de algo que só a fase 2 faz deixa a fase 1 vermelha. Sem o marcador, o pendente tem que falhar pelo motivo do critério (passo 6).
Fonte: Jest, API; pytest, "How to use skip and xfail"; `ralph.sh`, gate 2.

3 Instale e faça *no seu repositório*

1 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou antes? Atualize:

Terminal · em qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

2 Instale o /plan e os três agentes no seu projeto.

Cria `.claude/commands/plan.md`, `.claude/agents/specifier.md`, `clarifier.md`, `planner.md` e `.claude/scripts/confere-phases.sh`. São os arquivos da gravação: os do BC Harness 0.2.0 (MIT), delegando pelo nome local. Nenhuma aula anterior precisa ser instalada antes. Se você já tem um arquivo com o mesmo nome, o seu fica intacto e o da aula vai para `.harness-lab-novo`.

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 4.3
```

3 Confira o frontmatter do comando e dos agentes.

O esperado é "Validation passed", sem o "with warnings", nas duas pastas.

Terminal · na raiz do seu projeto — shell

```
ls .claude/commands .claude/agents .claude/scripts
claude plugin validate .claude/commands
claude plugin validate .claude/agents
```

4 Garanta o AGENTS.md da aula 3.4.

Já rodou o `/bc-harness:ai-context`? Pule este passo. Senão, instale o plugin (fica para o seu usuário, em todos os projetos):

Terminal — claude

```
claude plugin marketplace add beerandcodeteam/beer-and-code-harness
claude plugin install bc-harness@beer-and-code
```

E rode na raiz do projeto real (com `--adopt` se você já tem um `CLAUDE.md` ou `AGENTS.md` que ele não gerou). Ele não faz commit: revise com `git diff`.

Dentro do Claude Code · no seu projeto real — bc-harness

```
/bc-harness:ai-context --adopt
```

5 Veja um PHASES que passa, antes de gerar o seu.

O exemplo do cadastro da 1.5, escrito à mão no formato do planner. O `--sem-aceite` pula a comparação com o `FEATURE-ALVO.md` do seu projeto, que é de outra feature. Esperado: `fases: 2 · tasks: 4 · erros: 0` e `resultado: ok`.

Terminal · na raiz do seu projeto — shell

```
M=~/harness-lab-materiais/modulo-4-prova
E=$M/aula-4.3-spec-com-criterio-de-aceite/modelos/exemplo-cadastro
.claude/scripts/confere-phases.sh --sem-aceite "$E/PHASES.md"
```

6 Critérios com ID e um teste por critério (antes do /plan).

É aqui que "faz o cadastro" vira critério que um teste confere. Vem antes do `/plan` porque, com os pendentes já na suíte, o planner cria a task que tira cada marcador. Abra o Claude Code na raiz, com o `FEATURE-ALVO.md` lá, e mande os dois pedidos, nesta ordem. O molde dos critérios está em `modelos/modelo-criterios.md`; os pedidos inteiros, com o marcador de cada stack, em `modelos/pedir-os-testes.md`.

OS DOIS PEDIDOS (LARAVEL + PEST; TROQUE ARQUIVO E MARCADOR NA SUA STACK)

1. Reescreva os critérios de aceite do FEATURE-ALVO.md. Cada um com ID AC-NN, na forma QUANDO ... o sistema DEVE (ou SE ... ENTÃO ... DEVE / NÃO DEVE), e uma linha "Confere:" dizendo o que um teste mede. Duas leituras possíveis? Marque `[NEEDS CLARIFICATION]` em vez de escolher. Lista "- AC-NN", sem caixinha. Não mexa nas outras seções.

2. Escreva um teste Pest por critério do FEATURE-ALVO.md, num arquivo em tests/Feature/, com o ID no nome de cada teste. Rode cada um sozinho: o que já passa fica ativo; o que falha porque a feature ainda não existe fica com `->todo()`, com o corpo inteiro escrito. Não mude código da aplicação. No fim, rode a suíte inteira e mostre o comando e a saída.

Esperado: AC-01 a AC-NN no `FEATURE-ALVO.md` e a suíte verde, com os novos pendentes no placar. No exemplo do cadastro, `Tests: 2 todos, 2 passed` no Pest e `2 passed, 2 skipped` no pytest. Tire o marcador de um pendente uma vez e rode só ele: tem que falhar pelo motivo do critério. Devolva o marcador. Com a guarda da 4.2 instalada, ponha o arquivo revisado no `.claude/aceite.txt`: no loop, só o marcador poderá sair.

Tem ponto de fora (API, fila, e-mail)? Antes do `/plan`, cole no `FEATURE-ALVO.md` o bloco Pontos de fora de `modulo-4-prova/aula-4.2-o-teste-verde-que-mente/modelos/integracao-na-spec.md`.

7 Rode o /plan na sua feature-alvo.

Na mesma raiz, numa sessão nova (se a pasta `.claude/agents/` acabou de ser criada, feche e abra de novo), com o `FEATURE-ALVO.md` da 1.5. Ou em texto, como na aula: `/plan Quero incluir um contador de visitas aos posts do blog`.

Dentro do Claude Code — sessão

```
/plan FEATURE-ALVO.md
```

Prefere o do plugin? O mesmo pipeline, com os agentes do plugin:

Dentro do Claude Code · com o plugin BC Harness — bc-harness

```
/bc-harness:plan FEATURE-ALVO.md
```

8

Responda as quatro paradas.

Na parada 1, confira critérios, slug e tier. Na 2, abra a `SPEC.md` e procure palavra vaga e marcador; para fazer o clarificador rodar, diga que tem dúvida. Na 4, confira as fases. Respostas que funcionam:

Dentro do Claude Code · nas paradas do /plan — sessão

```
> Confirmo. Conteúdo da SPEC em português.
> Aprovo, mas tenho dúvidas: rode o clarifier antes de seguir.
> Re-delegue ao planner. Não edite o PHASES à mão.
```

ACRESCENTE NA PARADA 1

Com os pendentes do passo 6: "Para o planner: cada teste pendente precisa de uma task que tira o marcador, na fase que entrega o comportamento, citando o AC-NN." Com o `SchemaEstritoTest.php` da 4.2 no `.claude/aceite.txt`, mais: "Cite `tests/Feature/Ai/SchemaEstritoTest.php` no `Testes:` de uma task; se houver critério de contrato, na task dele." O passo a passo completo, parada por parada, está em `modelos/pedir-o-plan.md`.

Na parada 2, cada requisito tem esta cara: ID, forma, frase e uma linha `AC:` que dá sim ou não. Além das formas da seção 1, o especificador lista *Optional* (ONDE ..., para recurso que pode estar ligado ou não), raro nas SPECs reais. E aparece um rótulo fora da lista, `[Ubiquitous]`: requisito sem gatilho ("O sistema DEVE ..."). Na SPEC da gravação, RF-07, UI-01 e UI-02 saíram assim.

.spec/features/post-view-counter/SPEC.md (trecho) — saída real da gravação, para ler

```
- RF-02 [Unwanted]: If a view registration refers to a post that is not public
  (draft, scheduled, `published_at` in the future, soft-deleted or nonexistent),
  then the system shall not create or change any counter.
  - AC: Registro enviado para post `draft`, `scheduled`, futuro, soft-deleted ou
    id inexistente deixa a contagem total de visitas da base inalterada.
```

9

Confira o PHASES fora da sessão.

Mesmo script, do mesmo jeito, que o pré-voo do `loop.sh` roda na 5.1. Esperado: `resultado: ok` em cada feature. Com `.claude/aceite.txt` e `FEATURE-ALVO.md` na raiz, ele também exige cada `AC-NN` e cada arquivo do `aceite.txt` numa task, com o caminho escrito igual.

Terminal · na raiz do seu projeto — shell

```
ls .spec/features
C=.claude/scripts/confere-phases.sh
for p in .spec/features/*/PHASES.md; do "$C" "$p"; done
git status --short
```

O `aceite.txt` da 4.2 lista o `tests/Feature/Ai/SchemaEstritoTest.php`. Sem uma task que cite esse caminho, o script dá `NÃO PASSA` aqui e o pré-voo do `loop.sh` barra na 5.1. Não tire o arquivo do `aceite.txt` nem edite o PHASES: peça ao `/plan` para re-delegar ao planner com o pedido do passo 8.

Depois, commit numa branch de trabalho (`git switch -c` antes do primeiro commit), que é onde o loop do Módulo 5 roda: `FEATURE-ALVO.md`, os testes e o `.spec/` (o time revisa a spec no PR; quem não quer a spec no repositório põe `.spec` no `.gitignore`). Por fim, a lição de casa da aula: poste as suas tasks nos comentários.

O /plan *instalado*

Trechos para ler, não para copiar. O comando é um roteador fino: normaliza a entrada, para nas paradas, delega e confere os arquivos em disco. Todo o formato da SPEC e do PLAN mora nos agentes.

`.claude/commands/plan.md (trechos)` — instalado pelo instalar.sh · não copie do PDF

```
argument-hint: "<description | path-to-description-file>"
allowed-tools: Task, Agent, Read, Write, Glob, Grep, Bash, AskUserQuestion

| Step | Agent | Artifact |
| §5 | `specifier` | `.spec/features/[slug]/SPEC.md` |
| §6 (conditional) | `clarifier` | `SPEC.md` updated in-place |
| §7 | `planner` | `.spec/features/[slug]/PLAN.md` + `PHASES.md` (...) |
(...)
Present `summary`, the ACs (drafted ones flagged), `slug`, and `tier`. The developer
confirms or corrects. **Confirmed ACs become the source of truth for the SPEC** (...)
Do not delegate before confirmation.
(...)
Run when `grep -c '\[NEEDS CLARIFICATION\]' SPEC.md` > 0, OR tier is `complete`,
OR the developer reports doubts. Otherwise skip.
(...)
- **Delegate by local agent name** (`specifier`, `clarifier`, `planner`).
- **Never write application code.** The pipeline writes only under
  `.spec/features/[slug]/` (router itself only under `.handoff/`).
```

4 Arquivos que *reforçam a aula*

`.claude/agents/specifier.md`

O prompt do especificador, por completo. As formas (Event-Driven, State-Driven, Conditional, Unwanted, Optional), um ID por requisito, as palavras proibidas ("fast", "good", "adequate", "when possible") e a regra que mais importa: duas interpretações viram marcador, "never guess".

No repositório: modulo-4-prova/aula-4.3-spec-com-criterio-de-aceite/projeto/

`.claude/agents/clarifier.md`

O chato de propósito, em 55 linhas. Modo `analyze` (só lê e pergunta) e `resolve` (aplica as suas respostas e sobe a versão). Prioriza por risco de retrabalho: contrato, regra de negócio, caso de borda, premissa.

No repositório: modulo-4-prova/aula-4.3-spec-com-criterio-de-aceite/projeto/

`.claude/agents/planner.md`

As regras do PHASES que o loop lê: `## Phase N:`, um preâmbulo de leitura em cada fase (sessão nova não lembra nada), `Acceptance criteria:` obrigatório e checkbox sem aninhar.

No repositório: modulo-4-prova/aula-4.3-spec-com-criterio-de-aceite/projeto/

`.claude/scripts/confere-phases.sh`

A Prova aplicada ao plano: confere o PHASES fora da sessão e aponta a linha do defeito. Testado com o PHASES real da gravação (5 fases, 10 tasks, 0 erro) e com seis defeitos de formato.

No repositório: modulo-4-prova/aula-4.3-spec-com-criterio-de-aceite/projeto/

`modelos/exemplo-cadastro/`

O rascunho da 1.5 virando critério: `FEATURE-ALVO.antes.md` e `FEATURE-ALVO.md` com AC-01 a AC-04, forma e linha "Confere" (o molde está em `modelo-criterios.md`). Um teste por critério em Pest, Vitest e pytest, e o PLAN e o PHASES do cadastro.

No repositório: modulo-4-prova/aula-4.3-spec-com-criterio-de-aceite/modelos/

`modelos/pedir-o-plan.md`

O `/plan` na sua feature-alvo, parada por parada: o que conferir, o que responder e o que fazer depois. Junto, `pedir-os-testes.md`: os pedidos para escrever um teste por critério.

No repositório: modulo-4-prova/aula-4.3-spec-com-criterio-de-aceite/modelos/

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
`/plan` abre o modo de planejamento	A sessão não carregou o `plan.md`: outra pasta, ou já aberta ao instalar	Sessão nova na raiz, depois do `instalar.sh`. Ou `/bc-harness:plan`
"AGENTS context tree not found"	Sem `AGENTS.md` e sem `docs/agents/`	Rode o `/bc-harness:ai-context` (passo 4) e o `/plan` de novo
O `/plan` não acha o `specifier`	A pasta `.claude/agents/` nasceu com a sessão aberta	Feche e abra a sessão
PHASES com uma fase só	Até 3 critérios: tier `light`	Na parada 1: "Corrija o tier para standard"
`confere-phases.sh`: "'## Notas' entre duas fases" ou "task 3 da fase 2 sem 'Testes:'"	PHASES editado à mão, ou o planner saiu do formato	"Re-delegue ao planner." Não edite à mão
`confere-phases.sh`: "AC-02 está no FEATURE-ALVO.md e em nenhuma task"	Com `.claude/aceite.txt` e `FEATURE-ALVO.md` na raiz, cada `AC-NN` precisa de task	Peça ao planner a task que ativa o teste, citando o `AC-NN`

SINTOMA	CAUSA	CORREÇÃO
`confere-phases.sh`: **"tests/Feature/Ai/SchemaEstritoTest.php está no aceite.txt e em nenhuma task"**	Arquivo do `aceite.txt` sem task que cite o caminho. O pré-voo do `loop.sh` da 5.1 barra igual	"Re-delegue ao planner: cite o arquivo no `Testes:` de uma task" (passo 8)
`./ralph.sh: No such file or directory`	O ralph mora no cache do plugin e roda Codex por padrão	Espere o Módulo 5

6 Você sai *com*

- [ ] `claude plugin validate` passa sem aviso em `.claude/commands` e `.claude/agents`.
- [ ] `FEATURE-ALVO.md` com AC-01 a AC-NN e um teste por critério na suíte, verde, com os novos pendentes.
- [ ] `/plan` rodado na sua feature-alvo: `.spec/features/<slug>/` com `SPEC.md`, `PLAN.md` e `PHASES.md`.
- [ ] Na SPEC, cada requisito com ID e uma linha `AC:` que dá sim ou não, sem palavra vaga e sem marcador sobrando.
- [ ] `confere-phases.sh` no seu PHASES termina com `resultado: ok`, com uma task para cada `AC-NN` e para cada arquivo do `.claude/aceite.txt`.
- [ ] Critérios e testes commitados numa branch de trabalho, com o `.spec/` junto ou no `.gitignore`: árvore limpa para o Módulo 5.
- [ ] As suas tasks postadas nos comentários da aula.

PROMETIDO NA AULA

- ✓ **"Eu vou montar um comando para vocês"** (0:15): o `/plan`, em `projeto/.claude/commands/plan.md`. Instalado no passo 2 (p. 4).
- ✓ **"Vocês vão ter acesso a esse cara"** (7:28, o prompt do especificador): `projeto/.claude/agents/specifier.md`, passo 2 (p. 4) e seção 4 (p. 7).
- ✓ **"Nos materiais de apoio, tem aqui esses três agentes e o comando para você pegar, trazer para o seu projeto e rodar agora"** (16:35): `specifier.md`, `clarifier.md`, `planner.md` e `plan.md` pelo `instalar.sh 4.3` (passo 2), rodados no passo 7 (p. 4 e 5).
- ✓ **"(...) recupera ele aí"** (0:23, o `FEATURE-ALVO.md` da 1.5) e **"roda agora o plan dentro da sua feature-alvo"** (16:48): `/plan FEATURE-ALVO.md` no passo 7 (p. 5) e o exemplo antes e depois em `modelos/exemplo-cadastro/`.
- ✓ **"A gente vai ter um teste por critério"** (3:38): passo 6 (p. 5), `modelos/pedir-os-testes.md` e um teste por critério em Pest, Vitest e pytest em `modelos/exemplo-cadastro/`.

7 Glossário

Spec (SPEC.md)	O que a feature tem que fazer, com critérios que um teste confere. Escrita antes do código.
Critério binário	Frase que dá sim ou não: estado final que se mede, como provar e o que não pode mudar.
GEARS	A sintaxe que o especificador pede, derivada do EARS: gatilho (quando houver) mais "o sistema DEVE". O plugin não fixa palavra-chave nem idioma.
RIGID e FLEXIBLE	RIGID é o que não se negocia (o quê). FLEXIBLE é sugestão de implementação (o como).
AS IS e TO BE	Como o trecho do sistema é hoje e como fica, com o mesmo tipo de diagrama.
Marcador de dúvida	`[NEEDS CLARIFICATION]`: duas leituras possíveis. Quem resolve é você, nunca o modelo.
Tier	`light`, `standard` ou `complete`. Sai da contagem de requisitos e muda a profundidade do plano.
PHASES.md	O plano no formato do loop: `## Phase N:` e tasks com critério e testes.

8 Para *ir além*

- Best practices: "Brainstorm with Claude first" ("Interview me until you could write the spec") e as cinco perguntas antes de deixar o Claude rodar sozinho.
- Goal: "Write an effective condition", a régua do critério binário: "One measurable end state", "A stated check", "Constraints that matter".
- Skills: como um arquivo de `.claude/commands/` ganha nome e por que o seu `/plan` substitui o nativo.
- Harness design for long-running application development (Anthropic Engineering): o "sprint contract", combinar o que é pronto antes de escrever código.
- BC Harness: o README do `/plan`, os três agentes e o `ralph.sh` que o Módulo 5 usa.

Próxima: *Aula 4.4 · E2E mínimo*

Cada critério tem o seu teste. Mas e o fluxo inteiro, do jeito que o usuário usa? Um teste ponta a ponta do caminho principal, no mesmo comando do juiz.